

Teaching/Project Demo (35 mins)

You are developing an advanced AI-augmented programming curriculum that culminates in a comprehensive final project. This project encompasses the complete software development lifecycle from design to deployment, requiring you to build, test, and launch a sophisticated system to the cloud.

You will demo this project on your interview day. Specifically:

Live Demo

1. Showcase your database design, including any required data types, data constraints, entity relationship diagram, and/or Data Definition Language (DDL) and Data Manipulation Language (DML) for storing all parsed data from a CVS file. Please be as detailed as possible.
2. Showcase your working software code, design diagrams, and both frontend and backend functionalities.
3. Showcase your cloud-hosted web application and its functionalities.
4. Showcase your Python-based console application and its functionalities.
5. You will be asked to leverage any AI-augmented approaches to update and deploy your software code.
6. You may freely include any additional teaching methods, tools, and mediums to demonstrate your project to a diverse audience.

You will refer to the following Project walkthrough to guide you through the requirements.

Project

This document outlines the functional and non-functional requirements for a Student Grade Management System for Pasadena City College that provides dual access through web and console interfaces, incorporating data processing, secure access control, and cloud deployment capabilities.

Scope

The system supports:

- CSV data ingestion and processing
- Relational database storage and management

- Secure multi-user access via web and console applications from a computer on the Internet.
- Role-based access control (Admin, Teacher, Student)
- Cloud hosting and deployment on public platforms

Product Requirement

System Components

- Web Application: Secure user login and data visualization interface
- Console Application: Secure user login and direct database interaction via Python with secure connections
- Relational Database: Structured student information storage
- RESTful API: Secure communication between frontend and backend

Core Features

- Data ingestion from CSV files
- Data transformation and storage
- Role-based access control
- Secure web and console access
- Cloud hosting and deployment

User Access Control

Role	Read Access	Write Access	Delete Access	Modify Sensitive Fields
Admin	✓	✓	✓	✓
Teacher	✓	✓	✗	✗
Student	✓	✗	✗	✗

- Restrict modification of sensitive fields (Address, Phone Number)
- Secure authentication with username/password or OAuth integration

Technical Architecture

Data Processing

- CSV Ingestion: Read and parse comma-separated value text files with headers and multiple rows
- Data Transformation: Convert CSV data into structured database records with validation

- Database Storage: Store processed data in normalized relational database tables with referential integrity

Cloud Deployment

- Web Hosting: Deploy on AWS, Azure, or GCP with HTTPS secure communication
- API Integration: RESTful API endpoints for data operations with token-based security
- Database Schema: Entity Relationship Diagram showing Students, Courses, Assignments, Grades, and Users tables

Web Application

Landing Page:

- Login with username/password fields, remember me option, and forgot password link

Post-Login Experience:

- Secure HTTPS connection to remote database
- Display summary of student grades
- Role-based navigation menu:
 - Admin: Full dashboard access
 - Teacher: View/edit grades and assignments
 - Student: View own grades only

API Integration

- Expose RESTful API endpoints for:
 - Fetching student data
 - Updating grades
 - Managing users
- Secure API using tokens or OAuth2
- API documentation

Console Application

Landing Page:

- Login with username/password fields, remember me option, and forgot password link
- Built using Python only. You will involve OOP concept in the app.
- Application connecting via SSL/TLS only
- Same permission levels as web application
- Functionality: Display data, query specific records, update grades, export to CSV/JSON

Technology Stack

- Backend: Python (Django/Flask), Java, Node.js, or equivalent
- Frontend: HTML/CSS/JavaScript (React/Angular/Vue)
- Relational Database: MySQL, PostgreSQL, or equivalent
- Cloud Platform: AWS, Azure, or GCP
- Authentication: JWT or OAuth2

UML Diagrams

Sequence Diagram

- Login flow
- Data retrieval flow
- Grade update flow

Component Diagram

Show components:

- Web Server
- Database
- API Gateway
- Console App
- Authentication Service

Glossary

- CSV: Comma-Separated Values
- ERD: Entity Relationship Diagram
- RESTful API: Representational State Transfer Application Programming Interface
- SSL/TLS: Secure Sockets Layer / Transport Layer Security

Product Delivery at the Beginning of the Teaching Demo:

- Source code (must be opened with any text editor.)
- Usernames and Passwords created (a text file).
- Compress the source code and user list and submit via Zoom chat.